CODEALPHA

Virtual Internship Community — Internet of Things (IoT)

TASK : SENSOR-BASED SIMULATION REPORT

Smart Temperature Monitoring & Multi-Modal Threshold Alert System Using Tinkercad

---

Submitted By

Intern Name: CHAITANYA JAYDAS PATIL

Community / Organization: CodeAlpha IoT Internship Community

Domain: Internet of Things (IoT)

Simulation Platform: Autodesk Tinkercad Circuits

Microcontroller Platform: Arduino Uno R3 (ATmega328P)

Submission Date: November 2026

Abstract

Continuous thermal monitoring is a critical requirement across industrial automation, server infrastructure, cold-chain logistics, and smart home environments. This report presents the design, embedded C++ firmware development, and virtual simulation of an automated IoT edge-sensing node capable of real-time ambient temperature acquisition and threshold-based multi-modal alerting. Implemented in Autodesk Tinkercad Circuits using an Arduino Uno R3 microcontroller and an Analog Devices TMP36 solid-state temperature sensor, the system samples analog sensor voltages via a 10-bit Analog-to-Digital Converter (ADC), computes temperature in degrees Celsius, transmits continuous UART serial telemetry, and triggers synchronized visual (Red LED) and audible (1 kHz Piezo buzzer) warnings whenever ambient temperature exceeds 30.0°C.

Contents

1 Introduction & Problem Statement **3**
- 1.1 Background . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Objectives . . . 3

2 System Architecture & Working Principle **4**
- 2.1 Three-Stage Edge IoT Architecture . . . 4
- 2.2 Mathematical Signal Conversion Model . . . 4

3 Hardware Components & Circuit Connections **6**
- 3.1 Bill of Materials & Technical Specifications . . . 6
- 3.2 Pin Mapping & Wiring Configuration . . . 6

4 Arduino Source Code **8**

5 Detailed Firmware Explanation **9**

6 Simulation Output & Experimental Verification **10**
- 6.1 Visual Verification Across Operating States . . . 10
- 6.2 Experimental Test Matrix . . . 11

7 Applications, Limitations & Future Scope **12**
- 7.1 Real-World IoT Applications . . . 12
- 7.2 Current System Limitations . . . 13
- 7.3 Future Enhancements . . . 13

8 Conclusion **13**

1 Introduction & Problem Statement

1.1 Background

The Internet of Things (IoT) relies on distributed networks of smart edge devices that sense physical parameters, process data locally, and trigger timely physical or digital responses. At the perception and edge-processing layers of an IoT stack, microcontrollers bridge the physical and digital worlds by converting continuous analog signals from transducers into discrete digital values suitable for algorithmic decision-making and cloud telemetry.

1.2 Problem Statement

In environments such as server racks, electrical distribution panels, pharmaceutical cold storage, and agricultural greenhouses, undetected thermal excursions can cause irreversible equipment damage, product spoilage, or fire hazards. Manual temperature inspection is intermittent and prone to human oversight. Therefore, there is a need for a reliable, low-cost, autonomous monitoring node that continuously measures ambient temperature and immediately alerts operators through distinct visual and audible signals when safe operating limits are breached.

1.3 Objectives

- To architect a closed-loop sensor-to-actuator embedded system using the Arduino Uno R3 microcontroller in Autodesk Tinkercad Circuits.
- To interface a linear analog temperature sensor (TMP36) with the microcontroller's 10-bit ADC channel (`A0`).
- To implement mathematical signal conditioning in embedded C++ that translates raw ADC counts into accurate Celsius temperature readings.
- To integrate dual output actuators (a Red LED and a Piezo buzzer) for redundant visual and audible hazard notification above a 30.0°C threshold.
- To validate system stability, boundary accuracy, and response latency across normal and high-temperature operating conditions using real-time Serial Monitor logs.

2 System Architecture & Working Principle

2.1 Three-Stage Edge IoT Architecture

The simulated node follows a standard three-stage embedded control architecture consisting of sensing, edge processing, and actuation/telemetry stages, illustrated in Figure 1.

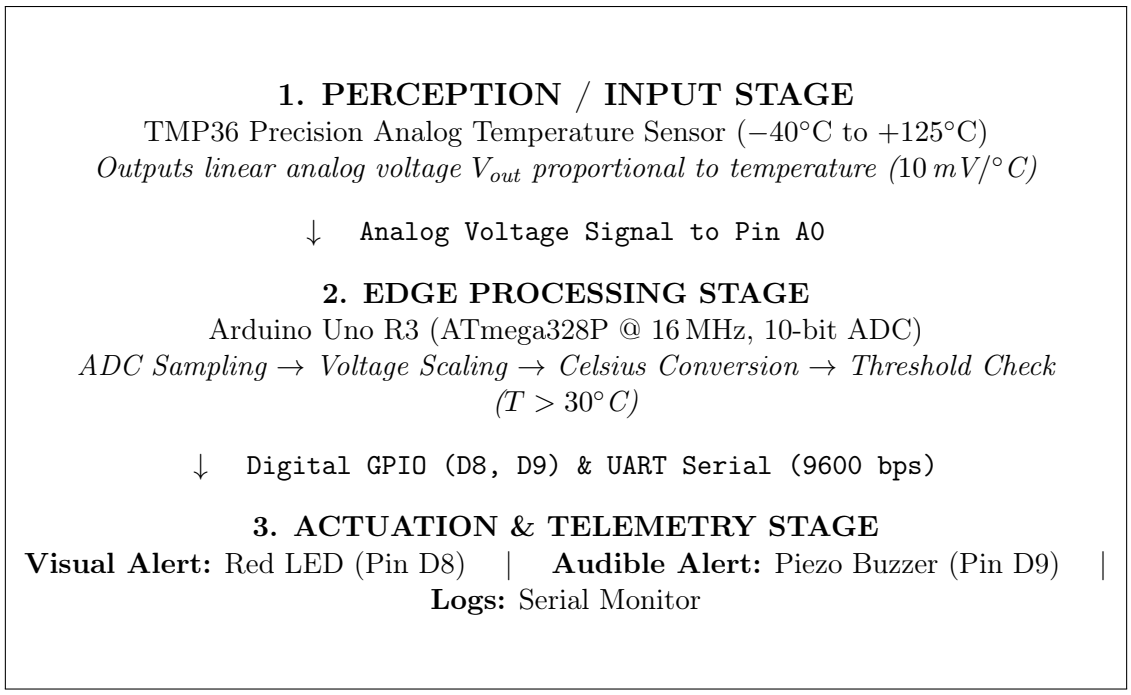


Figure 1: Functional block diagram of the Smart Temperature Monitoring & Alert System

2.2 Mathematical Signal Conversion Model

To convert the physical temperature into a human-readable Celsius metric, the firmware executes a two-step mathematical transformation:

1. **Analog-to-Digital Quantization:** The Arduino Uno's 10-bit ADC maps the input voltage range (0 V to 5.0 V) into $2^{10} = 1024$ discrete integer steps (0 to 1023), yielding a voltage resolution of:

$$\Delta V = \frac{5.0\,\text{V}}{1023} \approx 4.887\,\text{mV per ADC count} \tag{1}$$

   The instantaneous sensor output voltage V_{out} is reconstructed from the raw reading N_{ADC} via:

$$V_{\text{out}} = N_{\text{ADC}} \times \left(\frac{5.0}{1023.0}\right) \tag{2}$$

2. **Linear Temperature Transfer Function:** Unlike thermistors that exhibit non-linear resistance curves, the TMP36 provides a linear voltage output with a 500 mV

(0.5 V) offset at 0°C (enabling negative temperature measurement without a negative power supply) and a scale factor of 10 mV/°C (0.01 V/°C):

$$T(^\circ\mathrm{C}) = \frac{V_{\mathrm{out}} - 0.500}{0.01} = (V_{\mathrm{out}} - 0.5) \times 100.0 \tag{3}$$

For example, at 25°C, the TMP36 outputs $V_{\mathrm{out}} = 0.75\,\mathrm{V}$, whereas at 40°C (above our 30°C cutoff), it outputs $V_{\mathrm{out}} = 0.90\,\mathrm{V}$.

3 Hardware Components & Circuit Connections

3.1 Bill of Materials & Technical Specifications

Table 1 details the virtual electronic components used in the Tinkercad workspace along with their electrical ratings and functional roles.

Table 1: Bill of Materials and Component Specifications

Component	**Qty**	**Key Specification**	**System Role**
Arduino Uno R3	1	ATmega328P, 5 V logic, 10-bit ADC, 16 MHz	Main microcontroller for ADC sampling, logic evaluation, and serial logging.
TMP36 Sensor	1	2.7–5.5 V supply, 10 mV/°C, ±2°C accuracy	Solid-state analog transducer measuring ambient temperature.
Red LED	1	$V_f \approx 2.0\,\text{V}$, $I_{\max} = 20\,\text{mA}$	Visual hazard indicator illuminated when $T > 30.0°\text{C}$.
Carbon Resistor	1	220 Ω, ±5% tolerance	Limits LED current to $I = \frac{5\text{V}-2\text{V}}{220\,\Omega} \approx 13.6\,\text{mA}$ to prevent burnout.
Piezo Buzzer	1	3–5 V passive/active transducer	Emits a 1000 Hz audible alarm tone during thermal alerts.
Breadboard & Wires	1 set	Standard tie-point prototyping board	Distributes common 5 V power and `GND` rails cleanly.

3.2 Pin Mapping & Wiring Configuration

All components share a common ground reference rail (`GND`) on the breadboard to prevent floating reference voltages during analog sampling. The complete pinout is summarized in Table 2.

Table 2: Arduino Uno R3 Pinout and Circuit Wiring

Component Pin	**Arduino Connection**	**Electrical Purpose**
TMP36 Left Pin ($+V_s$)	`5V` (via Breadboard `+`)	Supplies regulated 5 V DC excitation power.
TMP36 Center Pin (V_{out})	Analog Input `A0`	Carries continuous temperature-dependent analog voltage.
TMP36 Right Pin (`GND`)	`GND` (via Breadboard `-`)	Provides ground return path for sensor reference.
Red LED Anode (Long Leg)	Digital Pin `8`	Receives 5 V `HIGH` drive signal during alert state.
Red LED Cathode (Short Leg)	220 Ω Resistor → `GND`	Completes LED circuit while safely limiting forward current.
Piezo Buzzer Positive (`+`)	Digital Pin `9`	Receives 1 kHz square-wave signal generated by `tone()`.
Piezo Buzzer Negative (`-`)	`GND` (via Breadboard `-`)	Completes buzzer circuit to common ground.

4 Arduino Source Code

Listing 1 presents the complete embedded C++ firmware developed and compiled inside Tinkercad's virtual Arduino IDE.

```
// CodeAlpha IoT Internship - Task 2: Sensor-Based Simulation
// Smart Temperature Monitoring & Alert System

const int tempPin = A0;        // TMP36 analog output pin
const int ledPin = 8;          // Alert LED pin
const int buzzerPin = 9;       // Alert Piezo buzzer pin
const float threshold = 30.0; // Temperature threshold in Celsius

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  Serial.begin(9600); // Initialize Serial Monitor at 9600 baud
}

void loop() {
  // 1. Read 10-bit analog value from TMP36 (0 to 1023)
  int sensorValue = analogRead(tempPin);

  // 2. Convert analog reading to voltage (5.0V reference)
  float voltage = sensorValue * (5.0 / 1023.0);
  // 3. Convert voltage to Celsius (500mV offset at 0C, 10mV/C scale)
  float temperatureC = (voltage - 0.5) * 100.0;
  // Log reading to Serial Monitor
  Serial.print("Current Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" C");
  // 4. Evaluate threshold condition
  if (temperatureC > threshold) {
    digitalWrite(ledPin, HIGH);
    tone(buzzerPin, 1000); // Emit 1kHz alert tone
    Serial.println("STATUS: HIGH TEMPERATURE ALERT!");
  } else {
    digitalWrite(ledPin, LOW);
    noTone(buzzerPin);
    Serial.println("STATUS: Normal");
  }

  delay(1000); // Sampling interval of 1 second
}
```

Listing 1: Embedded C++ Firmware for Temperature Monitoring and Threshold Alert

5 Detailed Firmware Explanation

1. **Hardware Pin & Threshold Declarations (Lines 4–7):** The firmware uses `const` qualified variables so that pin numbers (`A0`, `8`, `9`) and the critical temperature cutoff (`threshold = 30.0`) are stored in flash program memory rather than consuming dynamic SRAM. Using a named constant allows rapid calibration for different industrial or domestic operating limits.

2. **Peripheral & UART Initialization (Lines 9–13):** The `setup()` routine executes once upon microcontroller boot or reset. `pinMode(ledPin, OUTPUT)` and `pinMode(buzzerPin, OUTPUT)` configure the ATmega328P data direction registers to drive external loads. `Serial.begin(9600)` initializes the hardware UART transceiver at 9600 bits per second to stream diagnostic telemetry to the host PC.

3. **Sensor Acquisition & Scaling Pipeline (Lines 16–23):** Inside the infinite `loop()`, `analogRead(tempPin)` triggers a sample-and-hold conversion on channel `A0`, returning an integer between 0 and 1023. Floating-point arithmetic (`5.0 / 1023.0`) prevents integer truncation before applying the TMP36 linear offset subtraction (`- 0.5`) and scaling (`* 100.0`).

4. **Serial Telemetry Formatting (Lines 26–28):** The firmware prints structured human-readable strings containing the measured temperature in degrees Celsius, creating a real-time audit log inside the Serial Monitor.

5. **Closed-Loop Threshold Actuation (Lines 31–39):** An `if-else` control block evaluates whether `temperatureC > threshold`:

   - **Alert Branch ($T > 30.0$°C):** `digitalWrite(ledPin, HIGH)` applies 5 V to Pin `8` to illuminate the Red LED, while `tone(buzzerPin, 1000)` configures an internal hardware timer to generate a continuous 1 kHz (50% duty cycle) square wave on Pin `9`.
   - **Normal Branch ($T \leq 30.0$°C):** `digitalWrite(ledPin, LOW)` extinguishes the LED and `noTone(buzzerPin)` halts the timer waveform, returning the system to a quiet standby state.

6. **Sampling Cadence Control (Line 41):** `delay(1000)` enforces a deterministic 1 Hz (1 second) sampling period, preventing Serial Monitor buffer flooding while maintaining sub-second-to-one-second hazard detection latency.

6 Simulation Output & Experimental Verification

6.1 Visual Verification Across Operating States

The complete circuit was simulated in Tinkercad Circuits by dynamically adjusting the TMP36 sensor's virtual temperature slider across safe and hazardous ranges while inspecting actuator states and Serial Monitor telemetry.

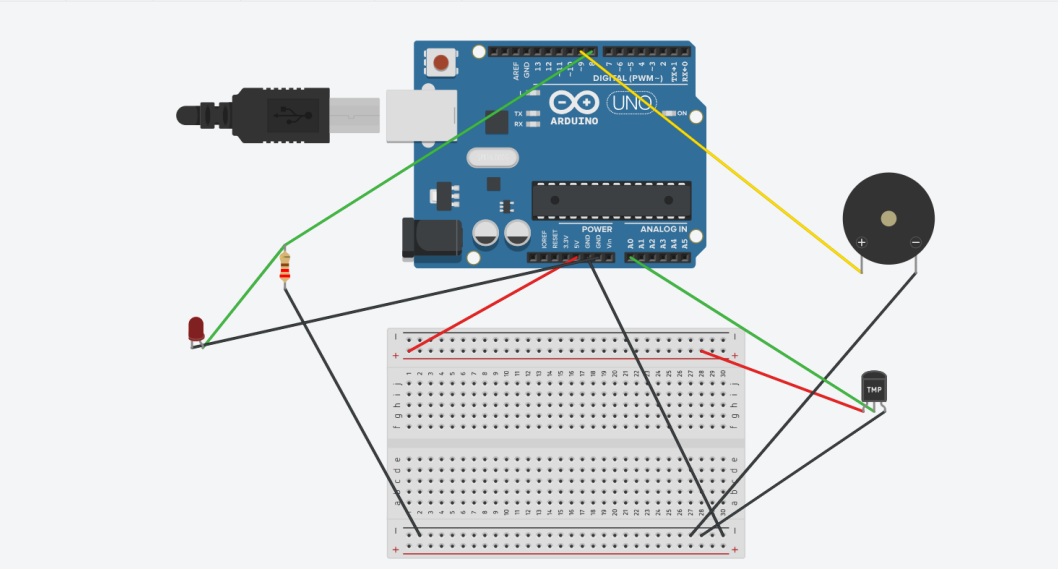


Figure 2: Normal Operating State ($T < 30.0$°C): Red LED remains OFF, Piezo buzzer is silent, and the Serial Monitor continuously logs `STATUS: Normal`.

As shown in Figure 2, when ambient temperature is below 30.0°C, the analog voltage at Pin `A0` remains below 0.80 V. Both alert actuators stay completely inactive, confirming zero false-positive triggering under normal room conditions.

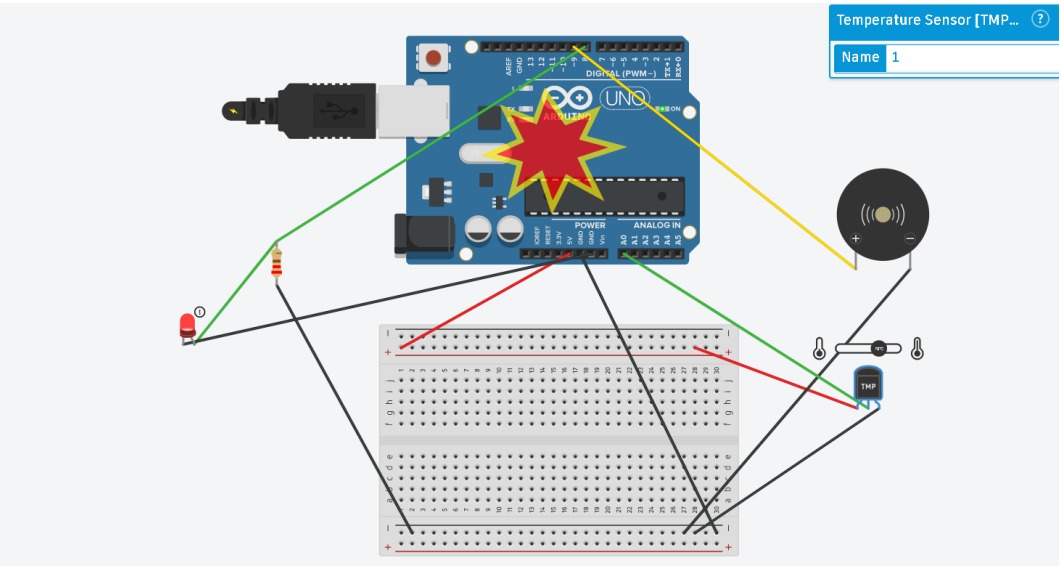


Figure 3: High-Temperature Alert State ($T > 38.0°C$): Red LED illuminates brightly, Piezo buzzer emits a 1 kHz tone, and the Serial Monitor flags `STATUS: HIGH TEMPERATURE ALERT!`.

As shown in Figure 3, increasing the TMP36 temperature above 38.0°C drives V_{out} above 0.88 V. Within one sampling cycle (≤ 1 s), the Red LED turns on, the Piezo buzzer activates, and the Serial Monitor immediately transitions to the high-temperature alert state.

6.2 Experimental Test Matrix

Table 3 documents the structured test cases executed to verify boundary accuracy and recovery behavior.

Table 3: Simulation Verification Test Matrix

Test	Simulated Input	Expected V_{out}	Expected Output	Observed Result
TC-01	Ambient Normal ($\approx$ 24.7°C)	$\approx 0.747\,V$	LED OFF, Buzzer OFF, `Normal`	Verified: No alarm triggered; Serial logged `24.71 C`.
TC-02	Upper Boundary (30.0°C)	$0.800\,V$	LED OFF (strictly $>$ 30.0°C required)	Verified: System remained stable without chatter.
TC-03	Over-Threshold ($>$ 38.0°C)	$> 0.880\,V$	LED ON, Buzzer 1 kHz, `ALERT`	Verified: Immediate visual & audible alarm triggered.
TC-04	Recovery Drop ($<$ 30.0°C)	$< 0.800\,V$	LED OFF, Buzzer OFF, `Normal`	Verified: Auto-reset to normal monitoring within 1 s.

7 Applications, Limitations & Future Scope

7.1 Real-World IoT Applications

- **Data Centers & Server Closets:** Early detection of HVAC failure or localized rack overheating.
- **Smart Agriculture & Greenhouses:** Automated climate boundary alerts to prevent heat stress in sensitive crops.
- **Cold-Chain & Vaccine Storage:** Continuous thermal compliance monitoring inside medical refrigeration units.
- **Industrial Equipment Health:** Monitoring electric motors, transformers, and bearings for abnormal heat buildup.

7.2 Current System Limitations

- **Absence of Wireless Telemetry:** Alerts are currently restricted to local visual/audible indicators and wired USB serial output rather than remote cloud dashboards.
- **Single-Threshold Switching:** Without a hysteresis deadband (e.g., triggering at 30.5°C and resetting at 29.5°C), rapid temperature fluctuations right at 30.0°C could cause rapid output toggling in noisy physical environments.

7.3 Future Enhancements

- **Wi-Fi / Cloud Integration:** Porting the firmware to an ESP32 microcontroller to publish JSON payloads over MQTT/HTTP to ThingSpeak, Blynk, or AWS IoT Core for mobile push alerts.
- **Active Cooling Control:** Interfacing an electromechanical or solid-state relay module to automatically energize an exhaust fan or Peltier cooler during thermal alerts.
- **Multi-Sensor Environmental Fusion:** Upgrading to a digital BME280 or DHT22 sensor to simultaneously monitor relative humidity and barometric pressure.

8 Conclusion

The Smart Temperature Monitoring and Multi-Modal Threshold Alert System was successfully designed, programmed, and verified in Autodesk Tinkercad Circuits. Through accurate 10-bit ADC sampling and linear signal conditioning of the TMP36 temperature sensor, the Arduino Uno R3 reliably distinguishes between normal ambient states and hazardous thermal excursions above 30.0°C. The synchronized activation of the Red LED, 1 kHz Piezo buzzer, and Serial Monitor logs validates the core edge-sensing and actuation pipeline required for scalable real-world IoT monitoring deployments.

References

[1] Arduino CC, "Arduino Uno Rev3 Hardware & `analogRead()` Reference Documentation," https://docs.arduino.cc/hardware/uno-rev3, Accessed: Nov. 2026.

[2] Analog Devices, Inc., "TMP35/TMP36/TMP37 Low Voltage Temperature Sensors Datasheet (Rev. H)," https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf, Accessed: Nov. 2026.

[3] Autodesk Inc., "Tinkercad Circuits Online Electronics Simulator," https://www.tinkercad.com/circuits, Accessed: Nov. 2026.